

SQL - Structured Query Language, ou Linguagem de Consulta Estruturada ou SQL

Criar uma base de dados (criar um banco de dados)

No mysql: create database locadora;

No postgresql: createdb locadora

Criar uma tabela chamada clientes e filmes

create table clientes

```
(  
    cli_codigo integer,  
    cli_nome   varchar(30),  
    cli_cidade varchar(20),  
    cli_sexo   char(1)  
);
```

create table filmes

```
(  
    fil_codigo integer,  
    fil_nome   varchar(30),  
    fil_genero varchar(15),  
    fil_preco  numeric(4,2)  
);
```

Inserir dados na tabela cliente;

**Insert into clientes (cli_codigo, cli_nome, cli_cidade, cli_sexo)
values (1,'Neri Neitzke', 'Carazinho', 'M');**

**Insert into clientes (cli_codigo, cli_nome, cli_cidade, cli_sexo,
cli_email)**

**values (1,'Neri Neitzke', 'Carazinho',
'M','neri@informaticon.com.br');**

**Insert into clientes (cli_codigo, cli_nome, cli_cidade, cli_sexo,
cli_email)**

```
values (2,'Lisiane Neitzke', 'Gurupi',  
       'F','lisiane@informaticon.com.br');  
Insert into clientes (cli_codigo, cli_nome, cli_cidade, cli_sexo,  
                      cli_email)  
values (3,'Giulia Neitzke', 'Machadinho',  
       'F','giulia@informaticon.com.br');  
Insert into clientes (cli_codigo, cli_nome, cli_cidade, cli_sexo,  
                      cli_email)  
values (4,'Gustavo Neitzke', 'Cascavel',  
       'M','gustavo@informaticon.com.br');
```

Ou

```
Insert into clientes values (1,"Neri Neitzke", "Carazinho", "M");
```

Visualizar, consultar, pesquisar, selecionar [SELECT]:

```
Select cli_codigo, cli_nome, cli_cidade, cli_sexo from clientes;
```

Ou

```
Select * from clientes;
```

Cláusula Order By

A cláusula Order By muda a ordem de apresentação do resultado da pesquisa e possibilita colocar também em ordem ascendente ou descendente.

Ex.:

```
Select * from clientes order by cli_nome;
```

Restrição where

Com where é possível restringir os dados (tuplas) que serão listados, ou seja, efetuar uma condição para que o mesmo apareça na listagem

Operadores lógicos

= igual a

> maior que

>= maior que ou igual a

< menor que

<= menor que ou igual a

Ex.: **Select * from clientes where cli_codigo >= 2;**

Select * from clientes where cli_codigo >= 2 and cli_codigo <= 5;

**Select * from clientes where cli_codigo >= 2 and cli_codigo <= 5
and cli_cidade = 'Carazinho';**

**Select * from clientes where cli_codigo >= 2 and cli_codigo <= 5
and cli_cidade = 'Carazinho' and cli_sexo = 'F';**

**Select * from clientes where cli_codigo >= 2 and cli_codigo <= 5
and cli_cidade = 'Carazinho' and cli_sexo = 'F' order by
cli_nome;**

Like = com padrão de caracteres

A linha abaixo indica que é para mostrar todas tuplas (registros) em que os nomes de clientes comecem com a letra G

Select * from clientes where cli_nome like 'G%';

A linha abaixo indica que é para mostrar todas tuplas (registros) em que os nomes de clientes terminem com a palavra Neitzke

Select * from clientes where cli_nome like '%Neitzke';

A linha abaixo indica que é para mostrar todas tuplas (registros) em que os nomes de clientes tenham a palavra Joao no meio (em qualquer lugar)

Select * from clientes where cli_nome like '%Joao%';

Betweenand = lista entre 2 valores

Select * from clientes where cli_codigo between 2 and 5;

É a mesma coisa que

Select * from clientes where cli_codigo >= 2 and cli_codigo <= 5;

Nas duas linhas acima, irá mostrar todas as tuplas em que o código seja maior ou igual a 2 e menor ou igual a 5, ou seja, entre 2 e 5

Select * from clientes where cli_codigo not between 2 and 5;

É a mesma coisa que

Select * from clientes where cli_codigo < 2 or cli_codigo > 5;
Nas duas linhas acima, irá mostrar todas as tuplas em que o código seja menor que 2 e maior que 5

In (..) – lista de valores pré definidos

Select * from clientes where cli_codigo in(2,4,6);

É a mesma coisa que

Select * from clientes where cli_codigo =2 or cli_codigo = 4 or cli_codigo = 6;

Nas duas linhas acima irá listar todas as tuplas em que o código seja 2 ou 4 ou 6

Select * from clientes where cli_codigo not in(2,4,6);

É a mesma coisa que

Select * from clientes where cli_codigo <> 2 and cli_codigo <> 4 and cli_codigo <> 6;

Nas duas linhas acima irá listar todas as tuplas em que o código seja diferente de 2 ou 4 ou 6

Manipulando Colunas das tabelas

Select cli_codigo 'Codigo do Cliente', cli_nome 'Nome do Cliente', cli_cidade 'Cidade do Cliente', clisexo 'Sexo' from clientes;

Distinct

A cláusula Distinct elimina duplicidades, não mostrando portanto dados duplicados como resultado de uma pesquisa.

Select distinct cli_cidade from clientes;

Primary key

Criando a chave principal, fazendo com que um determinado campo não possa ser repetido.

```
create table clientes  
(  
    cli_codigo integer not null,  
    cli_nome  varchar(30) not null,  
    cli_cidade varchar(20),  
    cli_sexo   char(1),  
    primary key (cli_codigo)  
);
```

Drop

Elimina uma tabela, seus dados, atributos e referências.

Ex.: drop table clientes

Alter

Este comando permite inserir/eliminar atributos nas tabelas já existentes.

Ex: alter table clientes add cli_email varchar (30);
alter table clientes drop cli_email; //apaga este atributo
alter table clientes drop column cli_email; //apaga no Oracle

modificar estrutura das colunas:

no mysql:

alter table clientes modify cli_email varchar (40);

no firebird:

alter table clientes alter column cli_email type varchar (40);

alter table clientes alter column cli_email to cli_correio;

no postgresql e Oracle:

alter table clientes alter column cli_email type varchar (40);

alter table clientes rename column cli_email to cli_correio;

Describe

Este comando permite visualizar a estrutura das tabelas

Ex: describe clientes;

update

Este comando permite alterar os dados da tabela

**Ex: update clientes set cli_sexo = 'M';
update clientes set cli_sexo = 'F' where cli_codigo =2;
update clientes set cli_cidade = 'Lajeado', cli_nome = 'Giulia
Neitzke' where cli_codigo =3;**

delete

Este comando permite excluir registros das tabelas

**Ex: delete from clientes; //exclui todos os registros da tabela
delete from clientes where cli_sexo = 'F';
delete from clientes where cli_cidade like 'C%';**

truncate

Este comando elimina de forma irreversível todos os dados da tabela

Ex: truncate table clientes;

Funções de Agregação

avg

Este comando calcula o valor medio de uma determinada coluna

Ex: select avg(cli_salario) as media_salarial from clientes;

count

Este comando conta as linhas de uma tabela

**Ex: select count(cli_codigo) from clientes;
select count(*) as quantidade_registros from clientes;**

sum

Este comando calcula a somatória de todos os valores de uma coluna

**Ex: select sum(cli_salario) as valor_total from clientes;
select sum(cli_salario) as valor_total from clientes where cli_sexo = 'F';**

min

Esta função sql retorna o valor mínimo encontrado em uma coluna

**Ex: select min(cli_salario) as valor_minimo from clientes;
select min(cli_salario) as salario_minimo from clientes where
cli_codigo in(1,3)**

max

Esta função sql retorna o valor máximo encontrado em uma coluna

**Ex: select max(cli_salario) as valor_minimo from clientes;
select max(cli_salario) as salario_minimo from clientes where
cli_codigo in(1,3)**

Funções escalares de Texto

upper

Esta função sql converte para maiúsculas todos os caracteres

**Ex: select cli_codigo,upper(cli_nome) as Nome,cli_cidade,
cli_sexo,cli_email,cli_salario from clientes;**

lower

Esta função sql converte para minúsculas todos os caracteres

Ex: `select cli_codigo,upper(cli_nome) as Nome,cli_cidade,
cli_sexo,lower(cli_email),cli_salario from clientes;`

Concatenar

concat (no MySQL)

Esta função sql no MySQL reúne em uma única sequência de caracteres, uma ou mais coulnas (atributos de tabela)

Ex: `select concat(cli_nome, cli_sexo) from clientes;
select concat(cli_nome, ' - ',cli_sexo) as 'Nome do Cliente e Sexo',
cli_cidade as Cidade from clientes;`

|| (no Firebird, PostgreSQL e Oracle)

Esta função Firebird reúne em uma única sequência de caracteres, uma ou mais colunas (atributos de tabela)

Ex: `select cli_nome || cli_sexo from clientes;
select cli_nome || ' - ' || cli_sexo as "Nome e sexo" from clientes;`

no Sql Server usa-se +

Esta função no sqlServer reúne em uma única sequência de caracteres, uma ou mais coulnas (atributos de tabela)

Ex: `select cli_nome + cli_sexo from clientes;
select cli_nome+' - '+cli_sexo as 'Nome e Sexo' from clientes;`

Índices

Índice é usado para agilizar pesquisas de seleção de dados nas tabelas. O Key é sinônimo de índice.

Ex: `show index from clientes;` *//mostra os indices da tabela*

create index ind_nome_cliente on clientes(cli_nome);

ou

alter TABLE clientes add Index ind_cliente_nome (cli_nome);

exclui o indice

no mysql, sql server = drop index ind_nome_cliente on clientes;

no firebird = drop index ind_nome_cliente;

group by (agrupar por..)

Agrupar dados

Para qualquer banco de dados

**Ex.: select cli_sexo as Sexo,
count(cli_sexo) as quantos
from clientes group by cli_sexo;**

ou

no Mysql

**select concat('Do sexo ',cli_sexo,' tem ', count(cli_sexo))
as 'Estatísticas sexuais'
from clientes group by cli_sexo;**

no Firebird e PostgreSQL e Oracle

**select 'Do sexo ' || cli_sexo || ' tem ', count(cli_sexo)
as "Estatísticas sexuais"
from clientes group by cli_sexo;**

no sql server

**select 'Do sexo ' + cli_sexo +' tem ', count(cli_sexo)
as 'Estatísticas sexuais'
from clientes group by cli_sexo;**

Resultado: Retorna quantos clientes são do sexo masculino e quantos são do sexo feminino.

```
select AVG(cli_salario) from clientes group by cli_sexo;
```

Neste exemplo acima irá retornar a media salarial agrupando por sexo

O exemplo abaixo funcionará perfeitamente no MySQL, no Firebird e PostgreSQL, você deverá trocar 'quantidade de pessoas' por “quantidade de pessoas”, ou seja, trocar apenas ' por “

```
select cli_sexo as Sexo,  
sum(cli_salario) as soma,  
AVG(cli_salario) as media,  
count(cli_sexo) as 'quantidade de pessoas'  
from clientes  
group by cli_sexo;
```

```
select cli_cidade as Cidade,  
cli_sexo Sexo,  
sum(cli_salario) as soma,  
AVG(cli_salario) as media,  
count(cli_sexo) as 'quantidade de pessoas'  
from clientes  
group by cli_cidade, cli_sexo;
```

Tabela de Cidades //para evitar redundância de dados

No MySql

```
CREATE table cidades
```

```
(  
    cid_codigo integer not null auto_increment,  
    cid_nome varchar(30),  
    primary key (cid_codigo)
```

);

No Firebird

CREATE table cidades

```
(  
    cid_codigo integer not null,  
    cid_nome varchar(30),  
    primary key (cid_codigo)  
);
```

Chave Estrangeira //foreign key

Efetua ligações entre as tabelas. Implementa integridade dos dados.

Alter table clientes

Add foreign key (cli_codcid) references cidades (cid_codigo);
Com constraint

Alter table clientes

Add constraint chave_estrang_cidade foreign key (cli_codcid)
references cidades (cid_codigo)
on delete RESTRICT
on update CASCADE;

Para apagar uma chave estrangeira

MySQL

alter table clientes drop FOREIGN key chave_estrang_cidade;

Firebird

alter table clientes drop constraint chave_estrang_cidade;

Union - Union All

Usado para unir o conteúdo de uma ou mais tabelas

```
select cid_codigo, cid_nome from cidades  
UNION  
select cli_codigo, cli_nome from clientes
```

Exemplo para unir dados de uma mesma tabela

```
select cli_codigo, cli_nome, cli_polegar as Digital_cliente  
from clientes where cli_polegar is NOT null  
union all  
select cli_codigo, cli_nome, cli_indicador as Digital_cliente  
from clientes where cli_indicador is NOT null  
union all  
select cli_codigo, cli_nome, cli_medio as Digital_cliente  
from clientes where cli_medio is NOT null  
union all  
select cli_codigo, cli_nome, cli_anelar as Digital_cliente  
from clientes where cli_anelar is NOT null  
union all  
select cli_codigo, cli_nome, cli_minimo as Digital_cliente  
from clientes where cli_minimo is NOT null;
```

commit

Efetua a gravação permanente de uma transação no banco de dados

Ex.: commit;

Auto Incremento

Preenche automaticamente o conteúdo de um campo inteiro, toda vez que um novo registro for inserido na tabela.

No MySQL = auto_increment

```
create table clientes  
(  
    cli_codigo integer not null auto_increment,  
    cli_nome varchar(30) not null,  
    cli_cidade varchar(20),  
    cli_sexo char(1),  
    primary key (cli_codigo)  
);
```

Ou, no caso da tabela já estar criada

```
ALTER table clientes MODIFY cli_codigo integer not null  
auto_increment;
```

Veja que para inserir você não precisa mais colocar o código.

Ex.: Insert into clientes (cli_nome, cli_cidade, cli_sexo, cli_email,
cli_salario, cli_codcid, cli_indicador)
values ('Selvino Neitzke', 'Lajeado',
'M','selvino@informaticon.com.br',50,1,'xzxzxzxzx');

No Firebird = Generator e Trigger

GENERATOR

O Firebird não possui um tipo de campo auto incremento. Mas você pode usar uma ferramenta chamada GENERATOR. Ele retorna um valor incrementado toda vez que você o chama. Veja abaixo como criar um generator

Ex.: CREATE GENERATOR gen_cidades_id;

Trigger

Triggers são chamadas automaticamente quando os dados da tabela a qual ela esta conectada são alterados

SET TERM ^

CREATE TRIGGER TR_CIDADES FOR CIDADES

ACTIVE BEFORE INSERT POSITION 0

AS

BEGIN

new.CID_CODIGO =gen_id(ge_cidades_id, 1);

END ^

Auto incremento no postgresql = serial

create table clientes

```
(  
    cli_codigo serial not null,  
    cli_nome  varchar(30) not null,  
    cli_cidade varchar(20),  
    cli_sexo   char(1),  
    constraint chave_primaria_bairro primary key (cli_codigo)  
);
```

Criação tabela no PostgreSQL

create table funcionario

```
(  
    fun_codigo serial not null,  
    fun_nome varchar(40),  
    fun_sexo char(1),  
    fun_salario decimal(10,2),  
    fun_codbairro integer not null,  
    constraint chave_primaria_funcionario primary key (fun_codigo),  
    constraint chave_estrangeira_codbairro foreign key (fun_codbairro) references bairro(bai_codigo)  
);
```

Domain e Check (Firebird, PostgreSQL)

Domain: Domains (domínios) são tipos de dados criados para que possam ser reaproveitados pelo usuário, assim, é fácil padronizar e facilitar a criação de tabelas.

```
CREATE DOMAIN "DM_CHAVE" AS INTEGER  
NOT NULL;
```

Check: Valida a entrada de dados em campos de nossas tabelas(valores mínimos, máximos, formatação etc)

```
CREATE DOMAIN "DM_CHAVE" AS INTEGER  
CHECK(VALUE > 0) NOT NULL;
```

Having para qualquer banco de dados

having: Esta clausula serve para restringir condicionalmente o retorno de uma instrução SQL através de uma função agregada.

```
Ex.: select fun_sexo, sum(fun_salario) from funcionario group by  
sexo having sum(fun_salario) >= 1000;
```

```
select fun_sexo, sum(fun_salario) from funcionario group by  
fun_sexo having count(fun_sexo) >= 2;
```

Join

join: usado quando temos que selecionar dados de duas ou mais tabelas.

```
Ex.: select fun_codigo, fun_nome, fun_salario, fun_sexo, nome as  
Cargo from funcionario join cargo on fun_codcargo = codigo;
```

É o mesmo que

```
select fun_codigo, fun_nome, fun_salario, fun_sexo, nome as Cargo  
from funcionario, cargo where fun_codcargo = codigo;
```

```
Ex2.: select cli_codigo, cli_nome, cli_salario, cli_sexo,  
        cid_nome as Cidade  
from clientes join cidades on cli_codcid = cid_codigo;
```

É o mesmo que

```
select cli_codigo, cli_nome, cli_salario, cli_sexo,  
        cid_nome as Cidade  
from clientes,cidades where cli_codcid = cid_codigo;
```

Projeto de Banco de dados

Modelagem de Banco de dados

Análise de Sistemas

MODELAGEM DE DADOS CONCEITUAL (Modelo ER)

Modelagem de Dados Conceitual é o início para o desenvolver e projetar banco de dados. É a descrição da estrutura de um banco de dados, independente de qual banco será usado (oracle, sql server, firebird, postgresQL, MySQL.....)

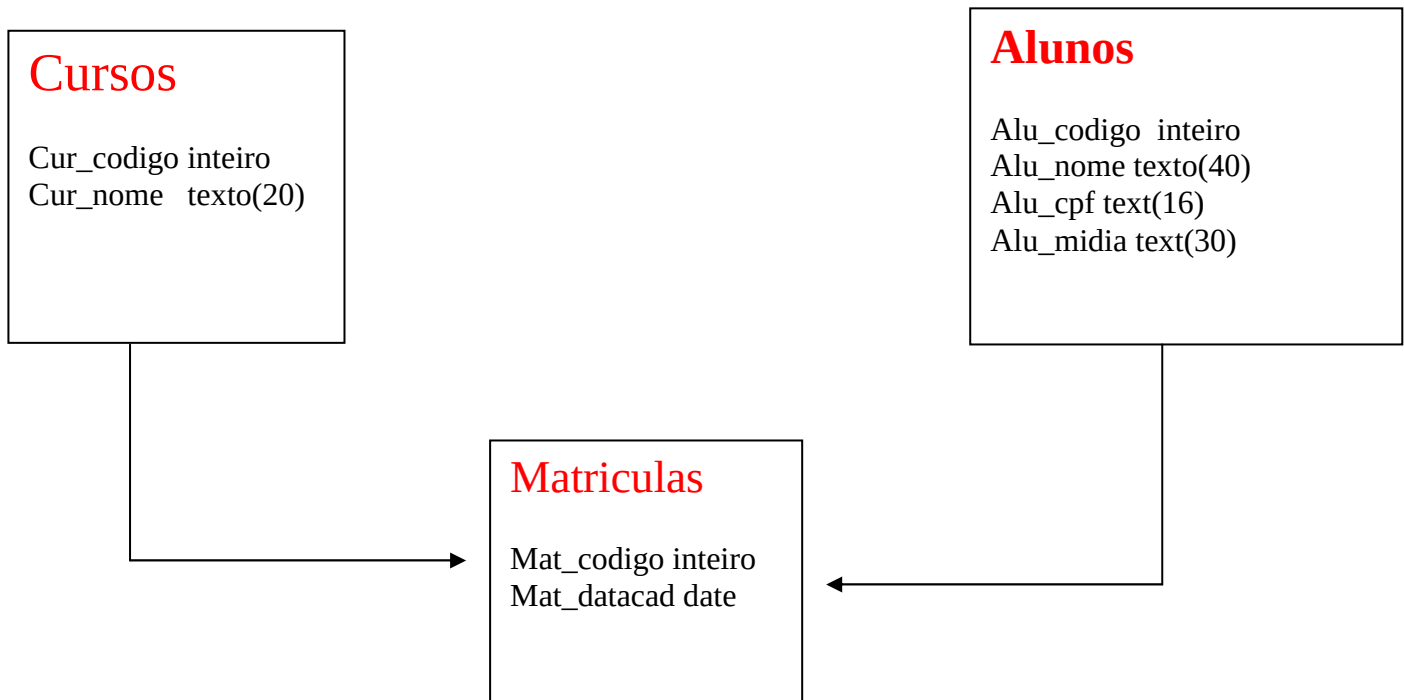
Importantíssimo:

- Faça uma análise super detalhada das informações e dados durante o estágio de modelagem de dados conceitual. Pois, se você tiver que mudar algo no seu projeto depois de ele estar pronto, a dor de cabeça poderá ser muito grande para fazer as alterações.

Componentes do Modelo Entidade-Relacionamento (modelo ER)

- Entidades - São os objetos onde as informações serão mantidas (tabelas)
- Atributos – São os dados das tabelas, relacionados a própria entidade ou a um relacionamento
- Relacionamentos – associações entre os atributos das entidades (**Efetua ligações entre as tabelas.**

Implementa integridade dos dados)



LIA - Listagem informal de atributos (rascunho)

= chave principal = primary key

@ = chave estrangeira = foreign key

- **PESSOA** (entidade para armazenamento das pessoas que podem ser tanto um aluno ou responsável, ou ambos)

pes_codigo # inteiro

pes_nome texto(40)

pes_codEndereco @(endereço)

pes_numero texto(10)

pes_complemento texto(10)

pes_codBairro @(bairro)

pes_codCidade @(cidade)

pes_cep texto(9)

pes_cpf texto(11)

pes_rg texto(18)

pes_foneres texto(16)

pes_fonecom texto(16)

pes_fonecel texto(16)

pes_datacad data

pes_datanasc data

pes_email texto(50)

pes_sexo texto(1) (M,F)

pes_estadocivil texto(1) (C,V,S,D)

pes_foto texto(50)

pes_digitalpolegar texto(40)

pes_digitalindicador texto(40)

pes_digitalmedio texto(40)

pes_digitalanelar texto(40)

pes_digitalminimo texto(40)

pes_observacao texto(80)

- **CIDADE** (entidade para armazenamento das cidades)
cid_codigo # inteiro
cid_nome texto(30)
cid_uf texto(2)
- **BAIRRO** (entidade para armazenamento dos bairros)
bai_codigo # inteiro
bai_nome texto(20)
- **ENDEREÇO** (entidade para armazenamento dos endereços)
end_codigo # inteiro
end_nome texto(40)
- **MIDIA** (entidade para armazenamento das mídias, exemplo: jornal, tv, rádio)
mid_codigo # inteiro
mid_nome texto(20)

MODELO ER

